

PostSpace — Blog Post Management App

October 2025 · Server-Side Web App · Node.js · Express · EJS

Anshumaan Sai Patnaik

Overview

PostSpace is a server-rendered web application for managing short blog-style posts. Each post has an author and a body, and the app supports the full set of create, read, update, and delete operations through a clean, RESTful interface. Pages are rendered on the server with EJS templates and styled with custom CSS.

The project's purpose was to implement a complete CRUD workflow end to end — routing, form handling, server-side rendering, and the HTTP semantics behind each action — on top of Node.js and Express. Data is held in an in-memory store, keeping the focus on request handling and REST conventions rather than database setup.

Technology Stack

Node.js	JavaScript runtime hosting the application server.
Express	Web framework handling routing, request parsing, and responses.
EJS	Server-side templating engine rendering the post list, detail, create, and edit views.
method-override	Middleware that lets HTML forms issue <i>PATCH</i> and <i>DELETE</i> requests, which browsers don't support natively.
uuid	Generates a unique identifier for each post.
HTML & CSS	Markup and custom styling for the rendered pages.

Architecture & Key Logic

The application follows a straightforward server-rendered architecture. Express is configured with three pieces of middleware: **urlencoded** body parsing to read submitted form fields, **method-override** to interpret the HTTP verb from a query parameter, and **static** file serving for the stylesheet. EJS is registered as the view engine, and each route renders a template with the data it needs.

Data model

Posts are stored in an in-memory array, each entry holding an **id**, a **username**, and the post **content**. Every post is assigned a UUID on creation, which is then used to look up, update, or remove that specific post across requests. Using a generated unique ID — rather than an array index — keeps references stable even as posts are added and deleted.

RESTful routing

Each operation maps to a dedicated route following REST conventions, with the post's UUID in the URL identifying the target record:

METHOD	PATH	PURPOSE
GET	/posts	List all posts.
GET	/posts/new	Show the create-post form.
POST	/posts	Create a new post and redirect to the list.
GET	/posts/:id	Show a single post.
GET	/posts/:id/edit	Show the edit form for a post.
PATCH	/posts/:id	Update a post's content.
DELETE	/posts/:id	Remove a post.

Handling update & delete from HTML forms

A notable detail of the build is supporting the *PATCH* and *DELETE* verbs. HTML forms can only send GET and POST, so the edit and delete forms submit as POST with a **?_method=patch** or **?_method=delete** query string; the method-override middleware rewrites the request to the intended verb before it reaches the route. This keeps the update and delete endpoints semantically correct REST routes rather than overloaded POST handlers.

Views

Four EJS templates make up the interface: a list view that loops over all posts and renders view, edit, and delete controls for each; a detail view; a create-post form; and an edit form pre-filled with the existing content. After a create, update, or delete, the server redirects back to the post list so the user always lands on an up-to-date view.